Reviewer Pack — Golden-Ratio Rotation Discrepancy of Truth Tables Excludes A…

Entry d329802a46f1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 10:17:03 UTC

Golden-Ratio Rotation Discrepancy of Truth Tables Excludes ACC^0[m]

Entry ID: `d329802a46f1` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 10:17:03 UTC

1. Conjecture

Field A (mathematical branch): Diophantine equidistribution: Erdős–Turán / Koksma rotation discrepancy of the golden-ratio Kronecker sequence (Weyl 1916, Erdős–Turán 1948, Schoissengeier bounds) applied as a P-uniform statistic on Boolean truth tables — essentially absent from circuit complexity **Field B** (complexity object): ACC^0[m] circuit size lower bounds for explicit P-time functions, in the Williams 2014 NEXP⊄ACC^0 / Nisan–Wigderson hardness-vs-randomness lineage

Statement:

> For f:{0,1}^n→{0,1} with truth table T_f∈{0,1}^{2^n} and φ=(√5−1)/2, define the golden-ratio rotation discrepancy D_φ(f) := max_{1≤N≤2^n} |∑_{i<N} (−1)^{T_f[i]} · e^{2πi φ i}| / √N. We conjecture (a) any f computed by depth-d size-s ACC^0[m] satisfies D_φ(f) ≤ (s·log m)^{c·d} for an absolute constant c, while (b) the explicit Beatty function f_φ(x):=⌊φ·int(x)⌋ mod 2 (computable in P) has D_φ(f_φ) ≥ 2^{n/4}/poly(n); together these imply f_φ ∉ ACC^0[m] of polynomial size and yield the natural property R(f):=[D_φ(f) ≥ (log 2^n)^{ω(1)}] which is poly(2^n)-constructive, useful, but non-large (random f has D_φ=O(√n) by Khintchine), threading Razborov–Rudich at the largeness axis only.

Rationale (proposer's reasoning):

> An irrational rotation by φ is spectrally orthogonal to every rational period dividing m, so any cyclotomic structure inherited by an ACC^0[m] truth table from Razborov–Smolensky low-degree GF(m)-polynomial approximators contributes only polylog to D_φ, whereas a Beatty word saturates D_φ by Weyl's theorem — exactly the asymmetry NW designs exploit when an irrational-rotation source fools mod-counting tests. Picking the largeness axis as the natural-proofs 'minimal violation' is principled because subexponential-time computability and ACC^0-uselessness are both desired by the Williams program, but largeness is the only Razborov–Rudich axiom not entangled with the Williams algorithmic-method machinery. No prior work links Erdős–Turán Beatty discrepancy of truth tables to ACC^0 size.

Taxonomy category: `NISAN_WIGDERSON_DESIGNS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c2eb7a8547dc5c65`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> At n=20 over 30 seeds: D_φ(f_φ) ≥ 20·median{D_φ(random)} on ≥25/30 seeds AND each of PARITY, MOD_3, MAJORITY, TRIBES has D_φ ≤ n^2=400 on ≥25/30 seeds. Falsified if any seed gives D_φ(f_φ)<√20≈4.47 or any ACC^0 candidate exceeds n^3=8000.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.88	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.70	SAFE	SAFE
ALGEBRIZATION	SAFE	0.70	SAFE	SAFE
KARP_LIPTON	SAFE	0.82	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `golden ratio Kronecker sequence discrepancy Boolean circuit lower bounds` - `Erdos-Turan discrepancy ACC^0 natural proofs Razborov-Rudich` - `Weyl equidistribution Beatty sequence circuit complexity hardness`

Top relevant hits considered: - [http://arxiv.org/abs/2504.19966v3] Quantum circuit lower bounds in the magic hierarchy - [http://arxiv.org/abs/2002.10956v2] Upper bounds on Kronecker coefficients with few rows - [http://arxiv.org/abs/1410.7087v3] Bounds on Kronecker and q-binomial coefficients - [http://arxiv.org/abs/0805.1385v3] Almost-natural proofs - [http://arxiv.org/abs/math/0302155v3] Generalized additive bases, Konig's lemma, and the Erdos-Turan conjecture - [http://arxiv.org/abs/0901.1649v1] The Erdos-Turan problem in infinite groups - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/2603.09958v1] Tetris is Hard with Just One Piece Type

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import math
import random
import sys

def golden_ratio_rotation_discrepancy(truth_table, phi):
    n = len(truth_table)
    s_N = 0
    max_disc = 0
    for i in range(n):
        s_N += (-1) ** truth_table[i] * (math.cos(2 * math.pi * phi * i) + 1j * math.sin(2 * math.
        max_disc = max(max_disc, abs(s_N) / math.sqrt(i + 1))
    return max_disc
```

```
def beatty_function(n):
    phi = (math.sqrt(5) - 1) / 2
    return [int(phi * i) % 2 for i in range(2**n)]

def parity_truth_table(n):
    truth_table = []
    for i in range(2**n):
        binary = bin(i)[2:].zfill(n)
        truth_table.append(int(binary.count('1') % 2))
    return truth_table

def mod_3_truth_table(n):
    truth_table = []
    for i in range(2**n):
        binary = bin(i)[2:].zfill(n)
        sum_bits = sum(int(bit) for bit in binary)
        truth_table.append(sum_bits % 3 == 0)
    return truth_table

def majority_truth_table(n):
    truth_table = []
    for i in range(2**n):
        binary = bin(i)[2:].zfill(n)
        ones_count = binary.count('1')
        truth_table.append(ones_count > n // 2)
    return truth_table

def and_of_or_tribes_truth_table(n):
    truth_table = []
    for i in range(2**n):
        binary = bin(i)[2:].zfill(n)
        ones_count = binary.count('1')
        if ones_count == 0 or ones_count == n:
            truth_table.append(0)
        else:
            truth_table.append(1)
    return truth_table

def uniform_random_truth_table(n):
    return [random.randint(0, 1) for _ in range(2**n)]

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [10, 14, 18, 20]
    results = []

    f_phi_truth_table = beatty_function(n)
    f_phi_disc = golden_ratio_rotation_discrepancy(f_phi_truth_table, (math.sqrt(5) - 1) / 2)
    results.append({"n": n, "f_phi_disc": f_phi_disc})

    for n in n_values:
        truth_tables = {
            "PARITY_n": parity_truth_table(n),
            "MOD_3_n": mod_3_truth_table(n),
            "MAJORITY_n": majority_truth_table(n),
            "AND-of-OR TRIBES_n": and_of_or_tribes_truth_table(n),
```

```
            "uniform_random": uniform_random_truth_table(n)
        }

        for name, truth_table in truth_tables.items():
            disc = golden_ratio_rotation_discrepancy(truth_table, (math.sqrt(5) - 1) / 2)
            results.append({"n": n, "name": name, "disc": disc})

    metric_name = "golden_ratio_rotation_discrepancy"
    metric_value = sum(result["f_phi_disc"] for result in results if "f_phi_disc" in result)
    instances_tested = len(results)
    conjecture_holds = all(result["f_phi_disc"] >= 20 * median_disc for result in results if "f_ph
                       all(result["disc"] <= n**2 for result in results if "name" in result and re
                           result["name"].startswith("MOD_3") or result["name"].startswith("MAJORI
                           result["name"].startswith("AND-of-OR TRIBES"))
    counterexample = "" if conjecture_holds else "f_phi_disc < 20 * median_disc"

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(seed) for seed in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = primes[:30]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f98e3adc.py", line 115, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f98e3adc.py", line 73, in run_trial
    f_phi_truth_table = beatty_function(n)
                                        ^
UnboundLocalError: cannot access local variable 'n' where it is not associated with a value
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed with an UnboundLocalError before any data was produced, so neither the support nor falsification condition can be evaluated. | next: Fix the scoping bug in beatty_function (pass n as a parameter or define it in scope) and rerun the 30-seed n=20 trial.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	209864
2	preregistration	claude_max	opus	0	0	7061
3	novelty	claude_max	opus	0	0	3389
4	novelty	claude_max	opus	0	0	9327
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14652
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13000
7	judge	claude_max	opus	0	0	3919

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 261212 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/d329802a46f1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d329802a46f1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d329802a46f1.tar.gz` (if generated)